

Intro to ORMs

Section Review

ORMs

- An **ORM** (Object-Relational Mapper) is a library that translates Python code into SQL and vice versa.
- The **ORM** is the communication layer between a programming language and the database.
- We picked SQLAlchemy as our ORM. SQLAlchemy is built on top of Pydantic and SQLAlchemy.

Database Models

- A database **model** is a Python class that maps to a complementary database table.
- Pydantic performs validation on the model object's attributes before the database write.
- SQLAlchemy also enforces additional validations that translate to the database layer (primary key, uniqueness, nullness, etc).

The Room Model I

- Define a Python class that inherits from **SQLModel** and add **table=True** to the subclass definition.
- Use **__tablename__** to customize the table name in the database.
- Import **Field** from **sqlmodel**, not from **pydantic**. The function supports Pydantic's validation parameters and adds database-specific ones.

The Room Model II

- Assign an attribute to **Field()** if it needs additional validation besides type checking.
- Set **primary_key=True** on the **id** column.
- Use **default** to provide a fallback value for a field.
- The **id** column is set to **int | None** so the model can represent the object before and after the database write.

The Database Engine

- An **engine** is an object that manages connections to a database.
- Use the **create_engine** function from SQLAlchemy to connect to a database (SQLite, Postgres, etc).
- The function accepts a string that designates the database type and location.
- SQLite saves all database content to a single file.

Creating the Database and Tables

- Define a function that invokes **SQLModel.metadata.create_all** and passes in the **engine** object.
- Make sure all database models have been imported/read by the time the function is defined.

FastAPI Lifespan

- FastAPI's class constructor accepts a **lifespan** function that runs when the app starts up.
- Define an **async** function decorated by **asynccontextmanager** from **contextlib**.
- Use **yield** to cede control to FastAPI.
- We invoked **create_db_and_tables**. When FastAPI starts up, SQLAlchemy will create the database file if it does not already exist.

Dependency Injection with Depends I

- The **Depends** function invokes a function and passes its return value as a parameter to a route handler.
- The pattern is identical to **Query**, **Cookie**, **Header**, and other constructs within FastAPI.
- Use **Annotated** in combination with the type, the **Depends** function, and the function to invoke.
 - **Annotated[Session, Depends(get_session)]**

Dependency Injection with Depends I

- **Dependency injection** (DI) is a design pattern where dependencies (objects the function needs) are passed as arguments rather than created inline.
- Dependency injection makes it easy to reuse dependencies across multiple functions.
- Dependency injection also makes it easy to swap a dependency when testing code.

The Session Dependency I

- A **session** is a temporary conversation with the database.
- We have one **engine** but create a session for each route handler execution.
- In the **get_session** function, we use the **yield** keyword to cede control to the route handler.
- Then, the context manager cleans up the session.

The Session Dependency II

- The **SessionDep** dependency/type variable uses **Depends** to invoke the **get_session** function.
- Declare **SessionDep** as a parameter type in any route handler. FastAPI will invoke **get_session** and provide the newly created session.
- The **with** context manager cleans up the session after the request is complete.

SQL Output

- We used the **Sqlite3 Editor** extension to view the contents of **database.db** within VSCode.
- Click **Schema** to see the SQL commands that were created by the ORM to create the tables.
- Add **echo=True** to the **create_engine** call to output SQL statements to your server logs.